

Ssail

Contenido

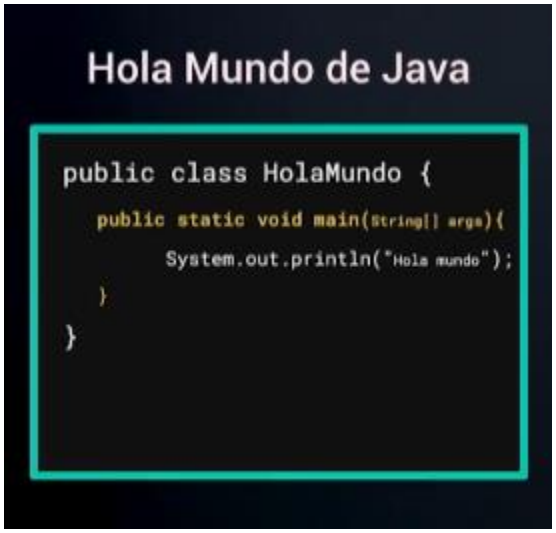
Introduccion a JAVA y Metodo MAIN PSVM 4
VARIABLES 8
VARIABLES CONSTANTES 9
CONVENCION CAMEL CASE 9
DATATYPES IN JAVA 10
Datos tipo int o Enteros y long 10
Datos con punto decimal, Float y Double 11
Datos de texto, Char y String 11
Datos lógicos, Boolean 12
Después de Java 10 Var 12
OPERADORES DE ASIGNACION 13
Operadores de incremento y decremento 13
OPERACIONES MATEMATICAS 15
CAST EN VARIABLES: ESTIMACIÓN Y EXACTITUD 16
CASTEO ENTRE TIPOS DE DATOS 17
ARCHIVOS .JAR 19
SENTENCIA IF 20
ALCANCE DE LAS VARIABLES Y SENTENCIA ELSE 21
OPERADORES DE EQUIDAD Y RELACIONALES 23
OPERADORES LOGICOS 23
SENTENCIA SWITCH 25
FUNCIONES 27
Implementación de funciones en java 28
JAVA DOCS 30
Activando JAVA DOCS 32
TAGS DE JAVA DOCS 33
Bucle Do While 34
OPERADOR TERNEARIO 36
Ciclo FOR 38
BREAK, CONTINUE and RETURN 38

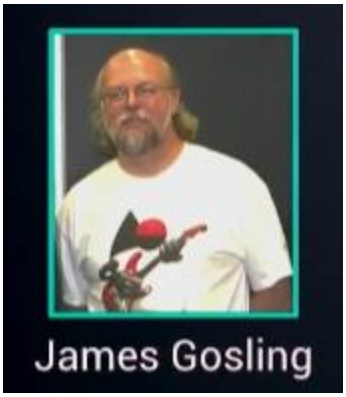
Arrays (Arreglos) 42
New 44
Aplicando arreglos 44
Declarando arrays de una dimensión 44
Declarando arrays de dos dimensiones 44
Declarando arrays de 3 en adelante dimensiones 45
Insertando datos en los arreglos; INDICES Y BUSQUEDA DE ELEMENTOS EN ARRAYS 45
Ciclos FOR anidados 48
FOREACH 50
Foreach anidados 50
Consejos For Anidados 51

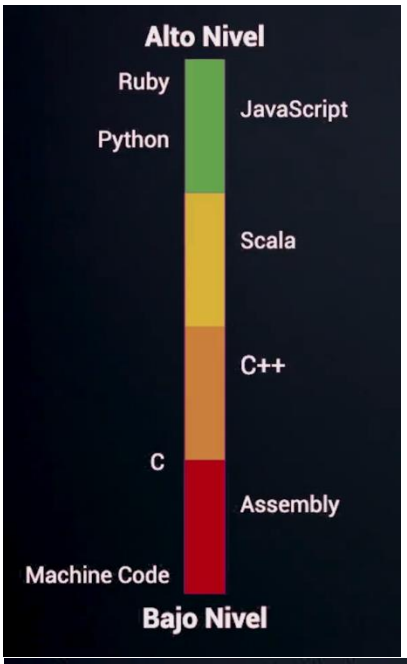
Introduccion a JAVA



JAVA SE “Java estándar edition” fue creado por James en 1991







Java es considerado un lenguaje de alto nivel y existen dos categorioas de java;

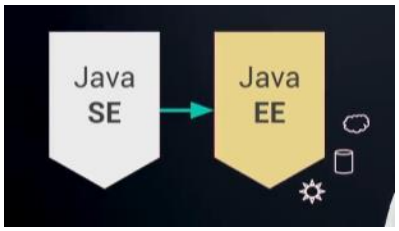
Java SE “Estándar Edition” y java EE “Enterprise Edition”

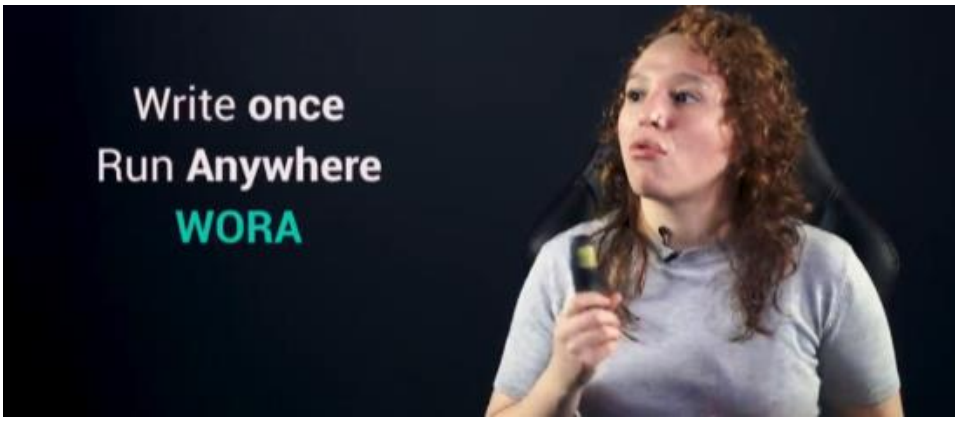




Java Enterprise esta dirigido a un uso mucho mas profesional, siendo el segundo escalón de Java SE

Sin embargo lo que se aprenda en Java SE se puede correr en java EE y en cualquier lugar, lo que convierte a Java en multiplataforma





Escribe una vez y corre en cualquier lugar



JDK Java Development Kit

- Es la maquina virtual que es lo permite que sea multiplataforma
- El compilador lo interpreta a algo que la maquina virtual pueda leer



- Es la base en la que podamos trabajar, integra herramientas para no trabajar desde cero.

Herramientas de JAVA

Versiones
La versión mas utilizada para construir proyectos web en servidores es Maiven siendo el doble de grande que Grable (Version que le hace competencia).

IDE



"Entorno de desarrollo integrado"

Podríamos programar solo utilizando un block de notas guardando el documento con la extencion .java para trabajar directamente sobre la terminal.
Sin embargo existen los IDE's que son los programas que nos ayudan a realizar el trabajo mas fácil.

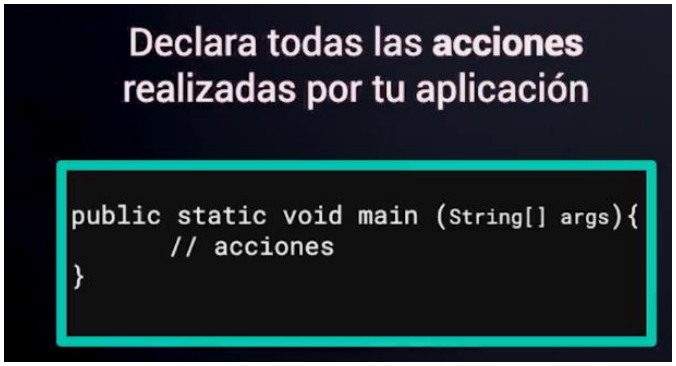


Oracle Recomienda NetBeans, sin embargo muchos programadores prefieren IJ

Ssail

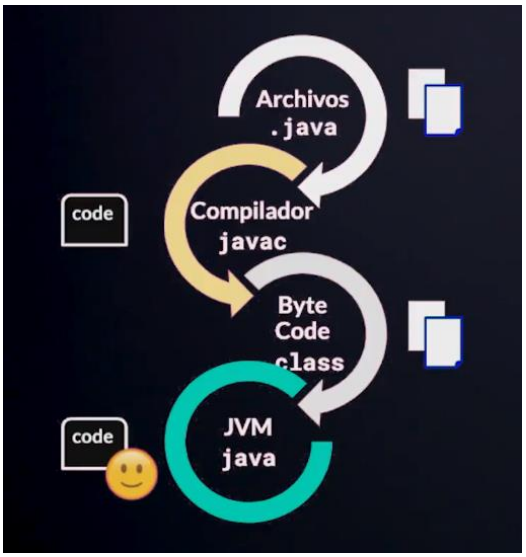
Método Main

El método Main es el punto de entrada de una aplicación de java.



En las acciones esta el proceso, el inicio de la aplicación.

Etapas de la programación en Java



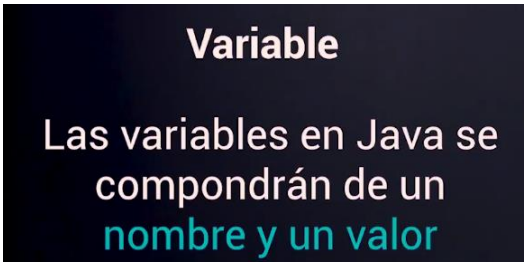
- Lo primero es el código fuente que es el documento con extensión .java
- El compilador se activa al presionar run o correr el programa, analiza el código y lo traduce a un lenguaje que la computadora lo pueda leer para que el programa cumpla con su objetivo.
- El lenguaje que la computadora puede leer es el bytecode
- Java Virtual Machine es el que crea el efecto multilenguaje



Por lo tanto Java es compilado e interpretado

VARIABLES






```
int celPhone = 55553333;
System.out.println(celphone);
System.out.println(celPhone);

String $countryName = "Spain";
String _backgroundColor = "Green";

String currency$ = "MXN";
String background_color = "BLUE";
```

Una buena práctica es siempre inicializar las variables
Ejemplo: estas convenciones son validas

Existen otro tipo de variables, variables de instancia que se definen a partir de las clases que se definen como objetos, variables estáticas como el que se encuentra en el método main, también se conocen como variables de clase y su alcance es mas global.

Actualizando variables

```
//2 horas extra $30 c/u
//Comida: $45

salary = salary + (30*2) - 45;
System.out.println(salary);

//Actualizando cadenas de texto
String employeeName = "Anahí Salgado";
employeeName = employeeName + " Díaz de la Vega";
System.out.println(employeeName);

employeeName = "Irene " + employeeName;
System.out.println(employeeName);
```


Ssail

VARIABLES CONSTANTES

Se pueden definir de estas maneras, se escriben en mayúsculas y contienen "_"

```
int POSITION = -5;
int MAX_WIDTH = 9999;
int MIN_WIDTH = 1;
```

o

```
public class NamingJava {
    public static void main(String[]
args) {
        final String NOMBRE_APELLIDO =
"Juan Perez";


System.out.println(NOMBRE_APELLIDO);
    }
}
```

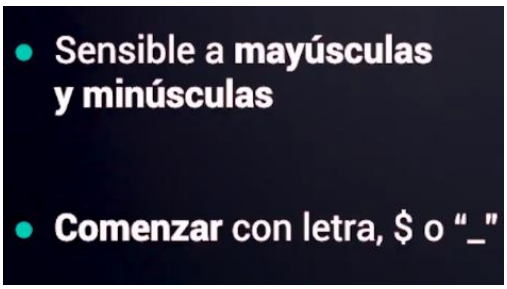
CONVENCION DE NOMBRES

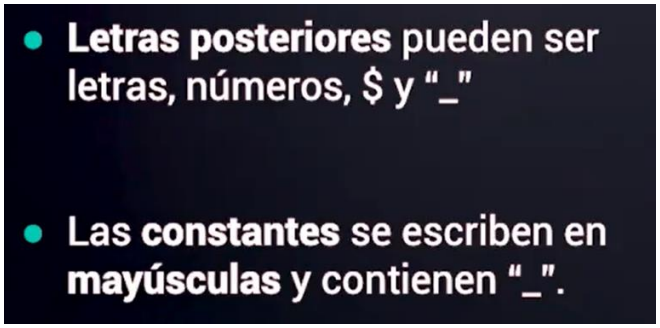
Camel case



UpperCamelCase => se usa para clases e interfaces

lowerCamelCase => se usa para métodos, variables y funciones

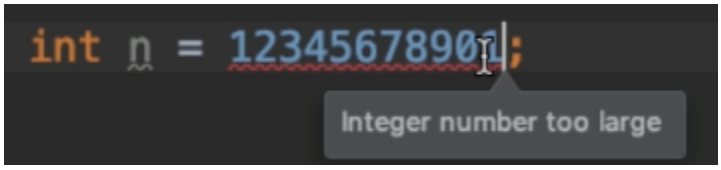




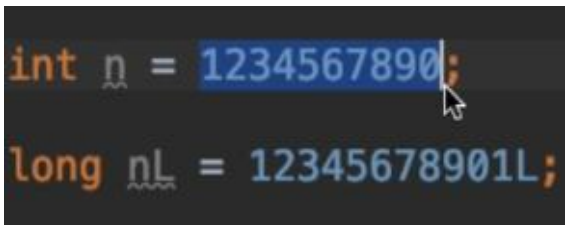
DATATYPES IN JAVA

Datos tipo int o Enteros y long

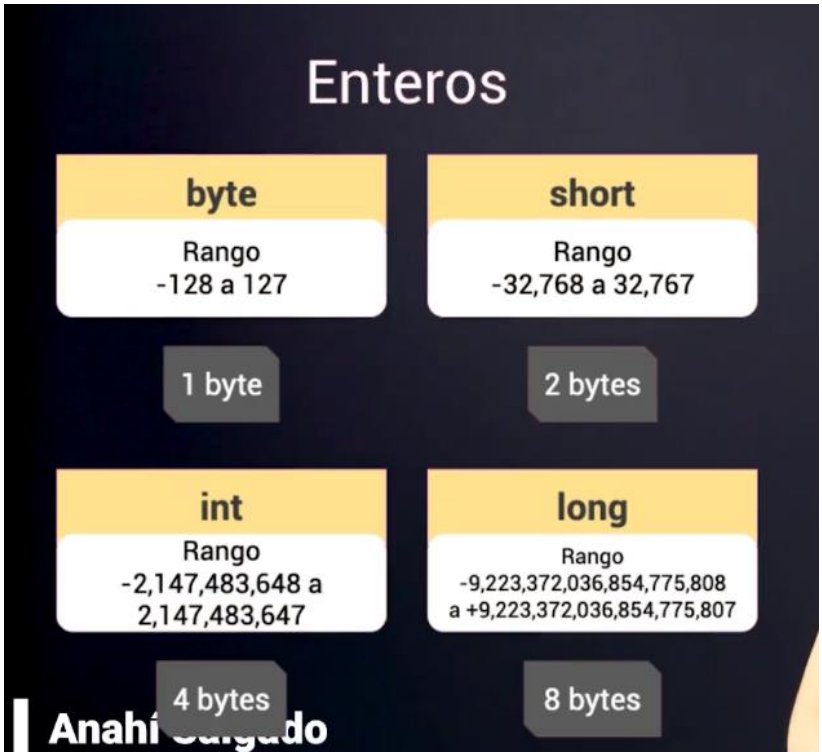
Solo se pueden tener 10 posiciones



Diferencia en declaración de datos int y long



Al declarar el valor de long este debe de llevar una L al final



Ssail

Datos con punto decimal, Float y Double

Declaracion de float and double



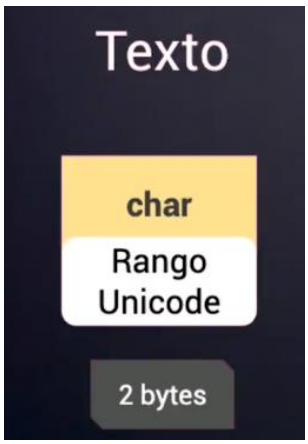

```
double nD = 123.456;
float nF= 123.456F;
```

Al declarar el valor de float este debe de llevar una F al final

Datos de texto, Char y String

Carácter char

```
char letter ='x';
```




```
import java.util.Scanner;
//@author ammgl
public class P15_Calculadora {
    public static void main(String[] args) {
        Scanner leer = new Scanner(System.in);
        char op;
        double n1, n2, res;
        System.out.print("Ingrese primer número:
        System.out.println("/) Division");
        System.out.print("Ingrese opción:\t");
        leer.nextLine();
        op = leer.nextLine().charAt(0);
        switch(op){
            case '+': res
        }
    }
}
```

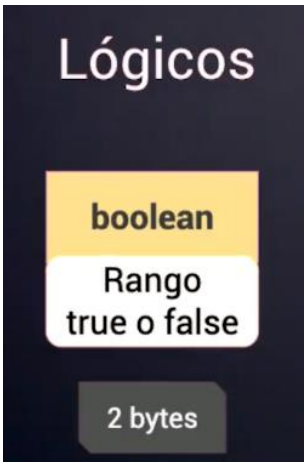

Ssail

Cadenas String

String ejemploCadena = “Luis Isaias”

```
String oper;   oper = leer.next();
```

Datos lógicos, Boolean



Después de Java 10 Var

Después de java 10 ya se puede omitir el keyword que hace referencia al tipo de dato que se colocara después de declarar la variable, sustituyéndola con la palabra var donde el compilador se encargara de inferir el tipo de dato.



var variable = valor;

// Creo que en NetBeans aun no se puede jsjsj xd 09/03/2022

// Sigue sin poder :v 20/09/2022

OPERADORES DE ASIGNACION



Operadores asignación

Operador	Aplicación	Desglose
+=	a += b	a = a + b
-=	a -= b	a = a - b
*=	a *= b	a = a * b
/=	a /= b	a = a / b
%=	a %= b	a = a % b

Operadores de incremento y decremento



Operador incremento decremento

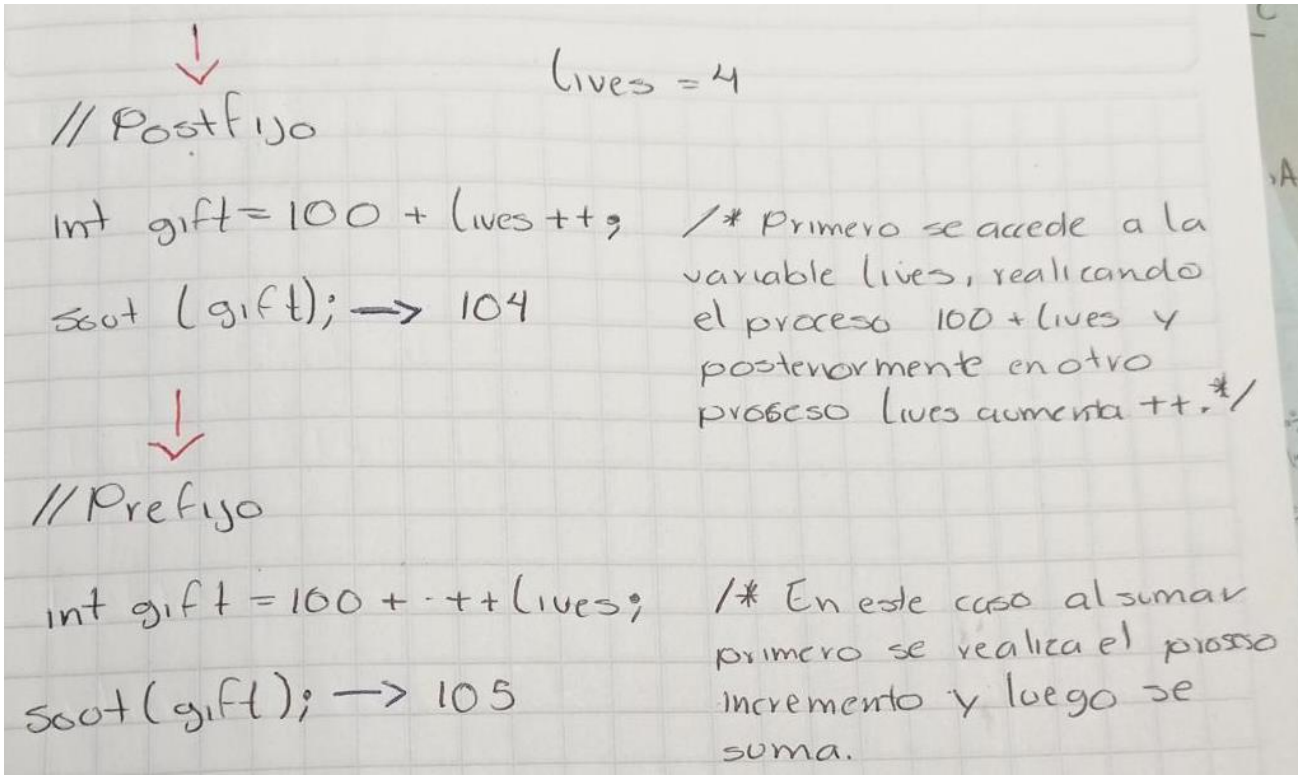
Operador	Nombre	Ejemplo	Desglose
++	incremento	i++	i = i + 1
--	decremento	i--	i = i - 1

Ssail

Prefijo y posfijo

Se puede usar de diferente manera ++i "Prefijo" y i++ "Posfijo", a nivel practicoes lo mismo pero a nivel logico tiene un significado





OPERACIONES MATEMATICAS

```java
public class MathematicOperations {
    public static void main(String[] args) {

        double x = 2.1, y = 3;

        //Devuelve un entero hacia arriba
        System.out.println(Math.ceil(x));

        //Devuelve un entero hacia abajo
        System.out.println(Math.floor(x));

        //Devuelve un número elevado a otro
        System.out.println(Math.pow(x,y));

        //Devuelve el numero mayor
        System.out.println(Math.max(x,y));

        //Devuelve la raíz cuadrada
        System.out.println(Math.sqrt(y));

        //Area de un circulo
        //pi * r2
        System.out.println(Math.PI * Math.pow(y,2));

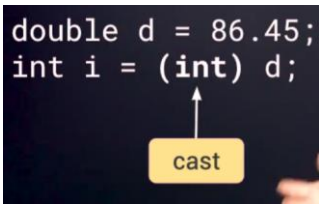
        //Area de una esfera
        //4*PI*r2
        System.out.println(4 * Math.PI * Math.pow(y,2));

        //Volumen de una esfera
        //(4/3)*pi * r3
        System.out.println((4/3) * Math.PI * Math.pow(y,3));

    }
}
```

CAST EN VARIABLES: ESTIMACIÓN Y EXACTITUD

En la programación hay situaciones donde necesitamos cambiar el tipo de dato de nuestras variables, esto lo conocemos como Cast.




```java
public class Casting {
    public static void main(String[] args) {

        // En un año ubicar 30 perritos
        // Cuántos perritos ubique al mes

        double monthlyDogs = 30.0/12.0;
        System.out.println(monthlyDogs);

        //ESIMACIÓN
        int estimatedMonthlyDogs = (int) monthlyDogs;
        System.out.println(estimatedMonthlyDogs);


        //Exactitud
        int a = 30;
        int b = 12;

        System.out.println((double) a/b);
    }
}
```

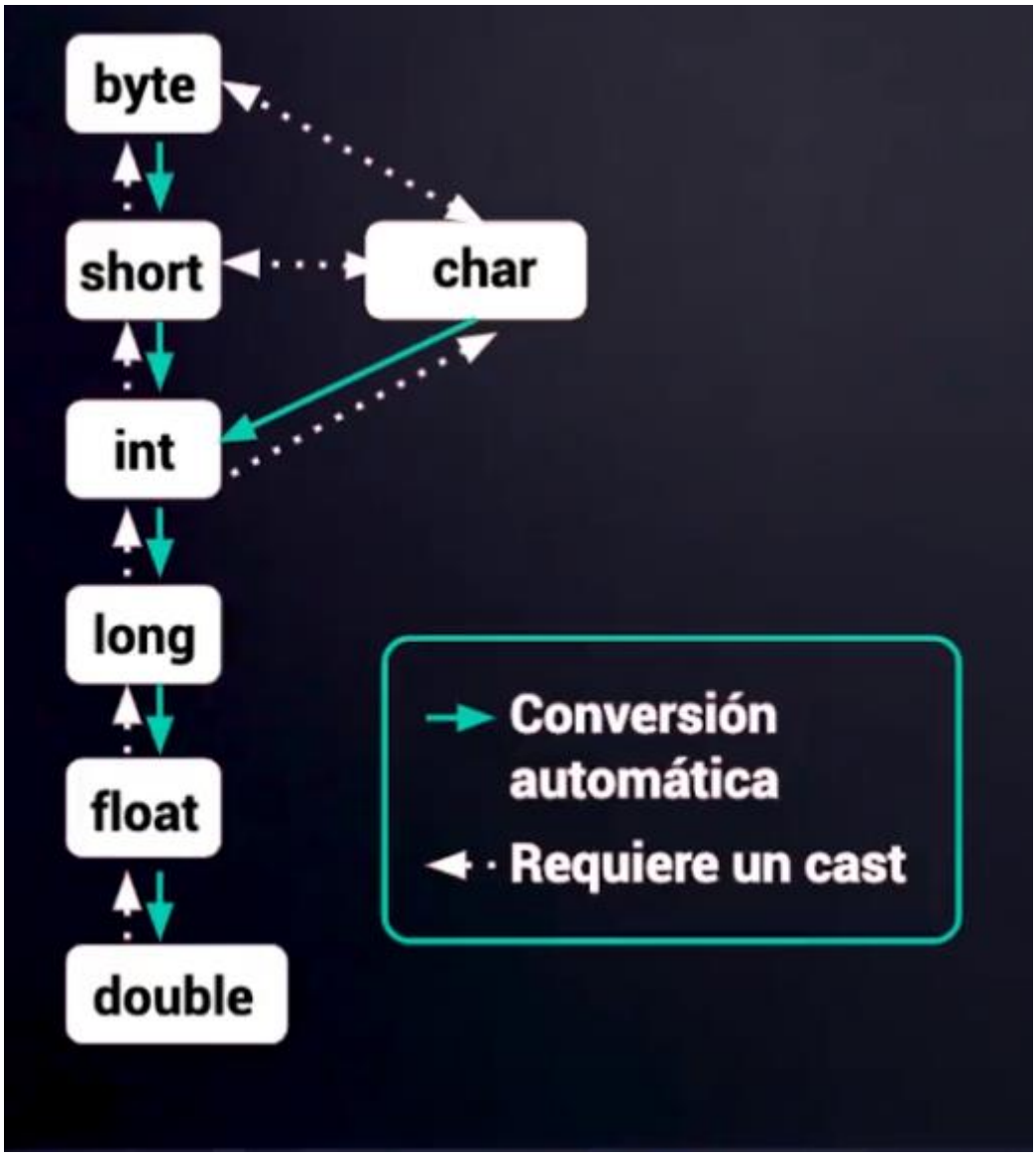
```
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding=UTF-8 -Duser.language=es -Duser.region=MX
2.5
2
2.5

Process finished with exit code 0
```

Se puede castear de esta manera también

```
//Devuelve la raíz cuadrada
System.out.println((int) Math.sqrt(y));
```

CASTEO ENTRE TIPOS DE DATOS



```
double c = a/b;
System.out.println(c);
```

```
Casting › main()
Casting
2.5
2
2.5
2.0
```

Sabemos que si lo casteo ya que el resultado es 2.0 sin embargo la división la trabaja con datos int por lo tanto el resultado es cerrado en int

```
double c = (double) a/b;
System.out.println(c);
    }
}
```

```
Casting › main()
Casting
2.5
2
2.5
2.5
```

Aquí obligamos a la división a realizarse en tipo double

POR LO TANTO: el casteo automático solo lo hace del resultado, no de los datos en el proceso.

```
char n = '1';
int nI = n;
```

```
Casting › main()
Casting
2
2.5
2.5
49
```

El carácter lo valida en ACII pero al cambiarlo a int lo devuelve en el valor decimal seleccionado

Caracteres ASCII imprimibles

32	espacio	64	@	96	`
33	!	65	A	97	a
34	"	66	B	98	b
35	#	67	C	99	c
36	$	68	D	100	d
37	%	69	E	101	e
38	&	70	F	102	f
39	'	71	G	103	g
40	(	72	H	104	h
41	)	73	I	105	i
42	*	74	J	106	j
43	+	75	K	107	k
44	,	76	L	108	l
45	-	77	M	109	m
46	.	78	N	110	n
47	/	79	O	111	o
48	0	80	P	112	p
49	1	81	Q	113	q

50	2	82	R	114	r
51	3	83	S	115	s
52	4	84	T	116	t
53	5	85	U	117	u
54	6	86	V	118	v
55	7	87	W	119	w
56	8	88	X	120	x
57	9	89	Y	121	y
58	:	90	Z	122	z
59	;	91	[	123	{
60	<	92	\	124	|
61	=	93	]	125	}
62	>	94	^	126	~
63	?	95	_		

En este caso si se requiere de un cast ya que el dato short es mas pequeño que los datos char

```
int a = 30;
int b = 12;

System.out.println((double) a/b);

double c = (double) a/b;
System.out.println(c);

char n = '1';
int nI = n;

System.out.println(nI);

short nS = (short) n;
System.out.println(nS);

    }
}
```

```
int nI = n;

System.out.println(nI);

short nS = (short) n;
System.out.println(nS);

    }
}
```

```
Casting › main()
Casting
2.5
2.5
49
49
```


Ssail

ARCHIVOS .JAR

Curso de introducción a Java SE

Los archivos JAR (*Java Archive*) son archivos de Java con el código compilado de los archivos `.class` y comprimido con el formato ZIP para que más adelante sean interpretados y ejecutados por la máquina virtual de Java (JVM).

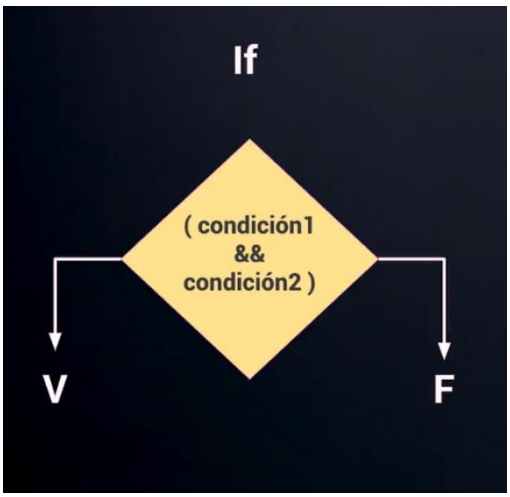
Para generar estos archivos podemos entrar a `File > Project Structure > Artifacts` y seleccionar la opción de `JAR > From modules with dependencies`. Luego de esto podemos compilar nuestro proyecto desde `Build > Build Artifacts > Build` y podremos nuestros archivos ejecutables en la carpeta `out/artifacts/`.

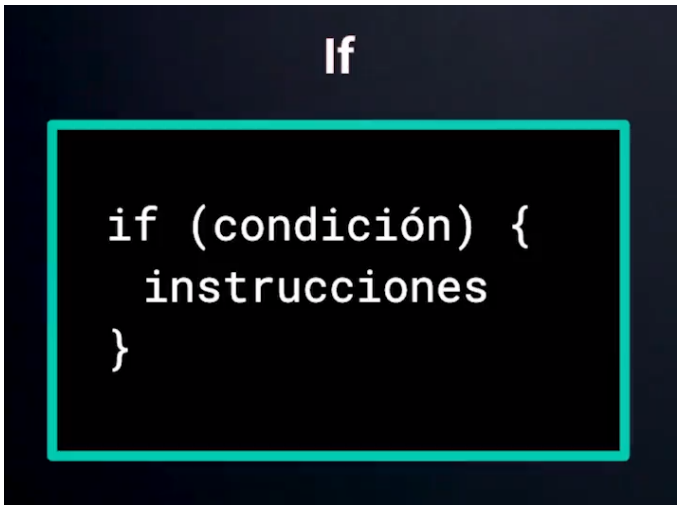
Clase 20

https://platzi.com/clases/1631-java-basico/21188-archivos-jar/

//alch no lo quiero hacer :v me vale vrg por ahora xdxxDXDxdXDxd

SENTENCIA IF





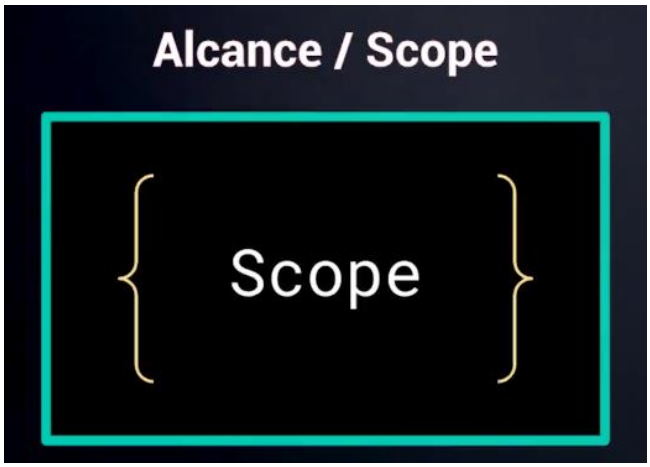

```java
import java.security.spec.RSAOtherPrimeInfo;

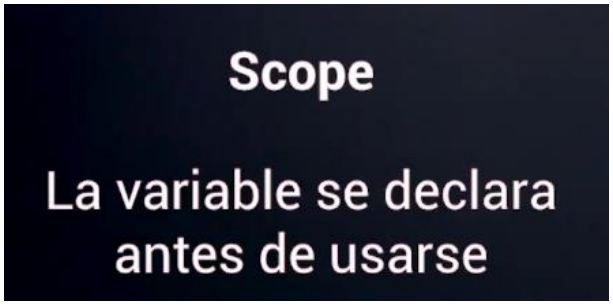
public class IfStatement {
    public static void main(String[] args) {
        boolean isBluetootEnabled = true;
        int fileSended = 3;

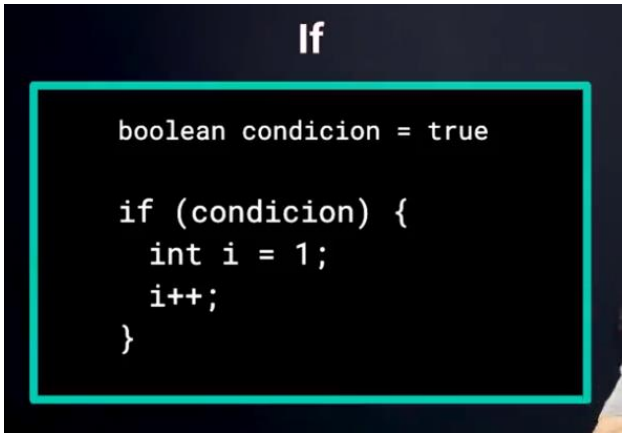
        if (isBluetootEnabled){
            //send file
            fileSended++;
            System.out.println("Archivo enviado " + fileSended);
        }
    }
}
```

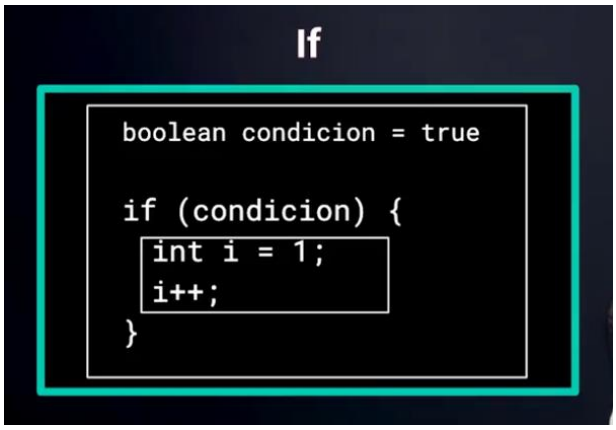

Ssail

ALCANCE DE LAS VARIABLES Y SENTENCIA ELSE

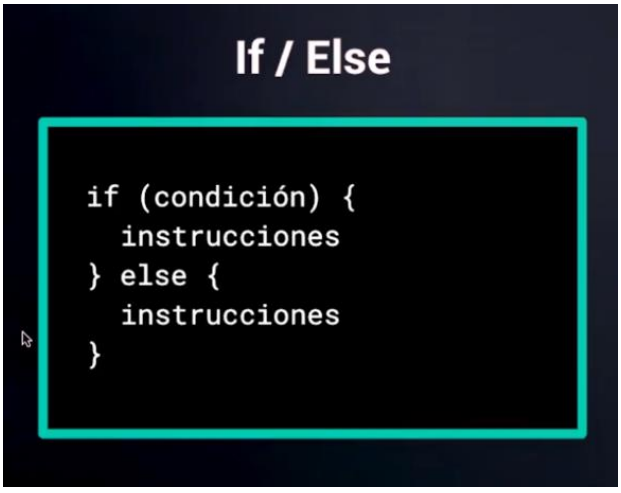








Las variables que definas dentro del scope solo tienen alcance dentro del mismo scope ya que no puedes accesar a ella fuera del scope



```java
import java.security.spec.RSAOtherPrimeInfo;

public class IfStatement {
    public static void main(String[] args) {
        boolean isBluetootEnabled = true;
        int fileSended = 3;

        if (isBluetootEnabled){
            //send file
            fileSended++;
            System.out.println("Archivo enviado " + fileSended);
            int i = 0;
            i++;
            System.out.println(i);
        } else {
            fileSended--;
            System.out.println("Please on Bluetooth for transfer");
            System.out.println(fileSended);
        }
        System.out.println();
        System.out.println(isBluetootEnabled);
        System.out.println(fileSended);

    }
}
```

OPERADORES DE EQUIDAD Y RELACIONALES

Devuelven valor falso o verdadero

Operador equidad

Operador	Nombre	Ejemplo
==	igualdad	a == b
!=	desigualdad	a != b

Operadores relacionales

Operador	Nombre	Ejemplo
<	Menor que	a < b
>	Mayor que	a > b
<=	Menor o igual que	a <= b
>=	Mayor o igual que	a >= b

OPERADORES LOGICOS

Operadores lógicos

Operador	Nombre	Ejemplo
&&	AND	a && b
||	OR	a || b
!	NOT	!a

Operadores lógicos

a	b	a && b
f	f	f
f	v	f
v	f	f
v	v	v

AND

a	b	a || b
f	f	f
f	v	v
v	f	v
v	v	v

OR

a	!a
f	v
v	f

NOT

```java
public class LogicOperations {
    public static void main(String[] args) {

        int a = 8, b = 5;

        //Operadores de asignacion
        System.out.println("a es igual de b? -> " + (a == b));
        System.out.println("a es diferente de b? -> " + (a != b));

        //Operadores Relacionales
        System.out.println("a es mayor de b? -> " + (a > b));
        System.out.println("a es menor de b? -> " + (a < b));
        System.out.println("a es mayor o igual de b? -> " + (a >= b));
        System.out.println("a es menor o igual de b? -> " + (a <= b));
    }
}
```

Ssail

If / Else if

```
if (condición) {
  instrucciones
} else if (condición) {
  instrucciones
} else {
  instrucciones
}
```

```java
public class LogicOperations {
    public static void main(String[] args) {

        int a = 8, b = 5;

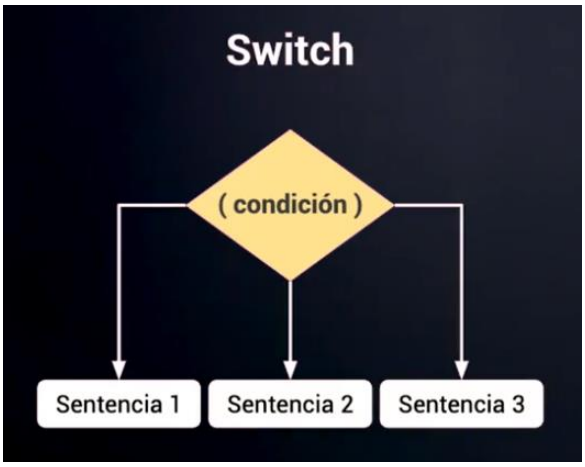
        //Operadores de asignacion
        System.out.println("a es igual de b? -> " + (a == b));
        System.out.println("a es diferente de b? -> " + (a != b));

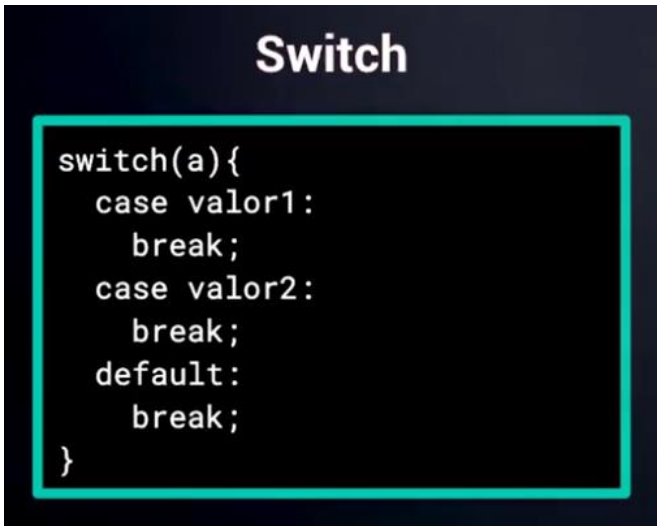
        //Operadores Relacionales
        System.out.println("a es mayor de b? -> " + (a > b));
        System.out.println("a es menor de b? -> " + (a < b));
        System.out.println("a es mayor o igual de b? -> " + (a >= b));
        System.out.println("a es menor o igual de b? -> " + (a <= b));

        if (a == b){
            System.out.println("a es igual de b");
        } else if (a != b){
            System.out.println("a es diferente de b");
        }else if (a > b){
            System.out.println("a es mayor de b");
        }else if (a < b){
            System.out.println("a es menor de b");
        }else if (a >= b){
            System.out.println("a es mayor o igual de b");
        }else if (a <= b){
            System.out.println("a es menor o igual de b");
        }
    }
}
```

Ssail

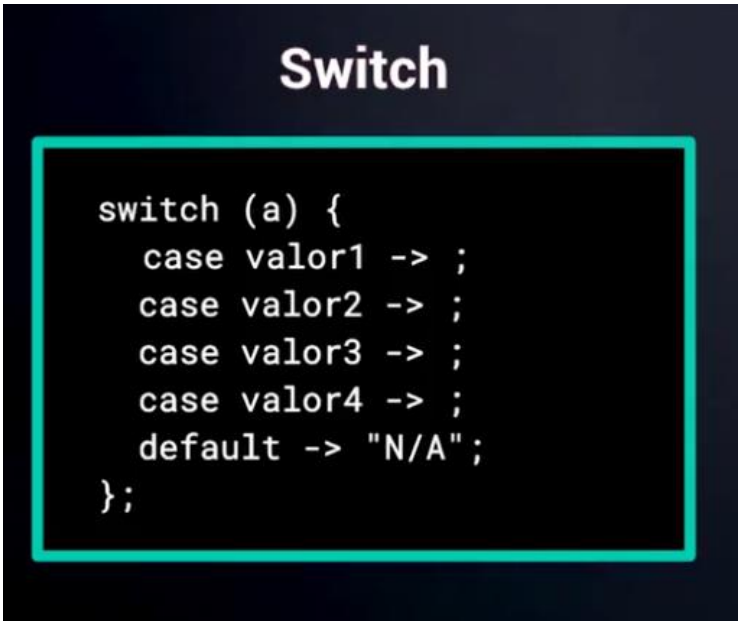
SENTENCIA SWITCH





Apartir de Java 12

Puede usarse asi, sin embargo para que funcione se tiene que realizar una serie de ajustes



En práctica …

```
switch (variable) {
        case caso1:
                instrucciones
                break;
        case caso2:
                instrucciones
                break;
        case caso3:
                instrucciones
                break;
        default:
                instrucciones
                break;
```

```java
public class SwichStatement {
    public static void main(String[] args) {

        String colorModeSelected = "Dark";

        switch (colorModeSelected){
            case "Light":
                System.out.println("Seleccionaste Light Mode");
                break;
            case "Night": //Ambar
                System.out.println("Seleccionaste Night Mode");
                break;
            case "Blue Dark":
                System.out.println("Seleccionaste Blue Dark Mode");
                break;
            case "Dark":
                System.out.println("Seleccionaste Dark Mode");
                break;

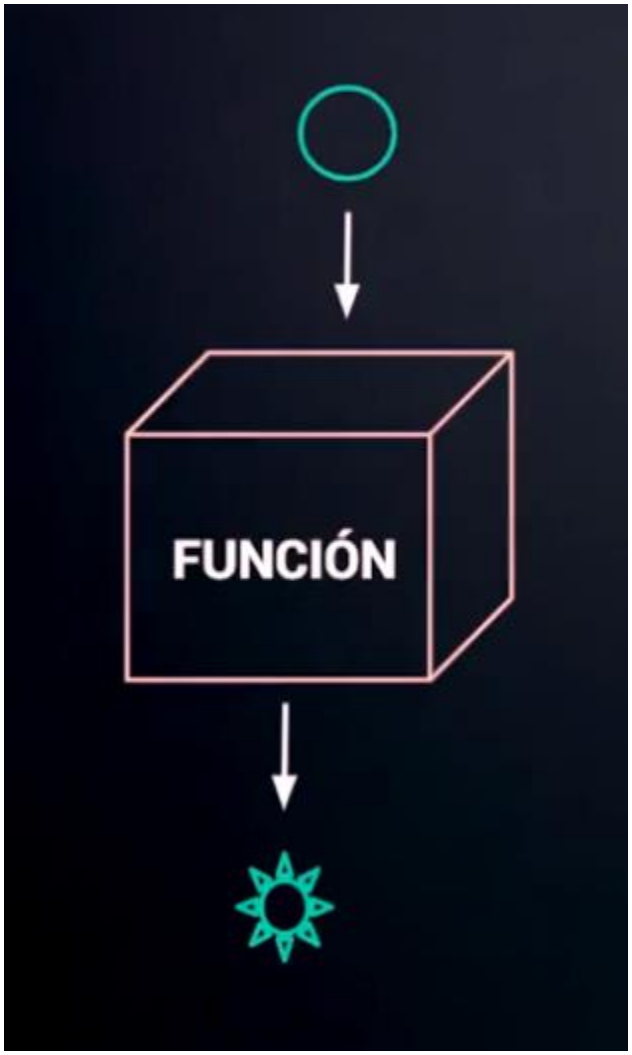
            default:
                System.out.println("Selecciona una opción");
        }

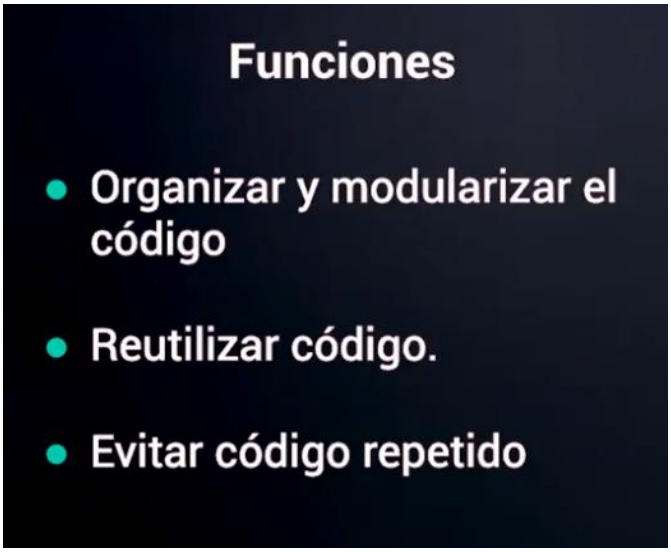
    }
}
```

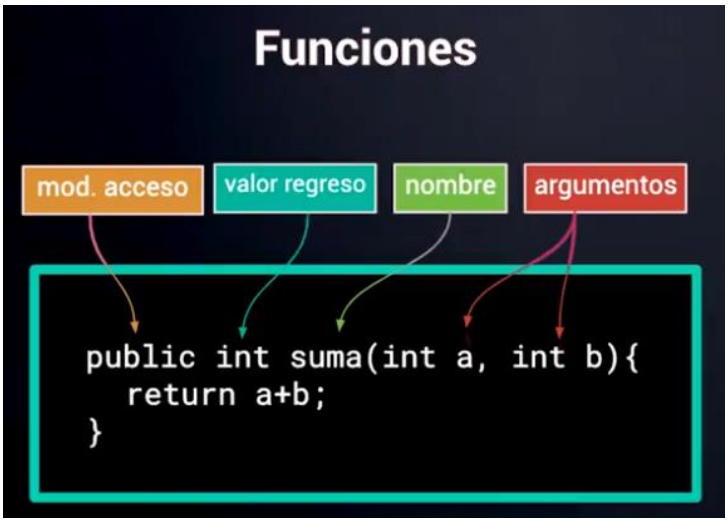
Ssail

FUNCIONES

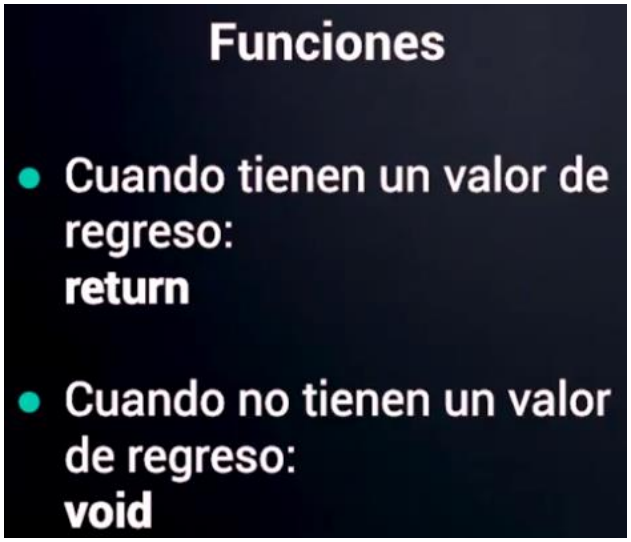
Las funciones siempre tienen una entrada y una salida

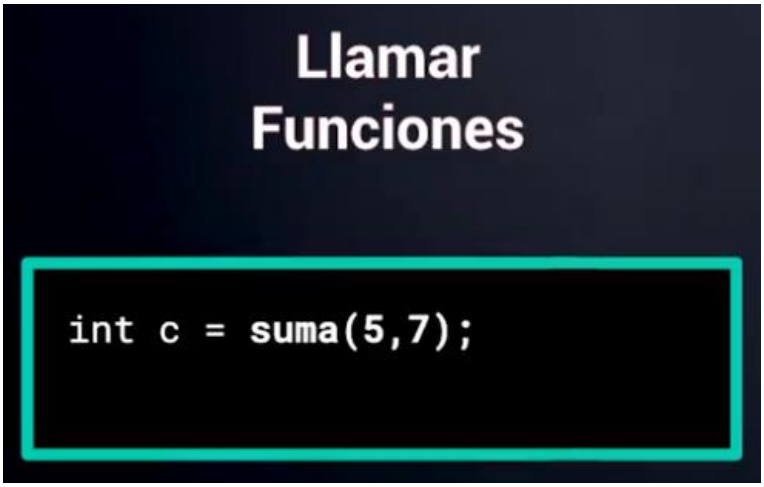






Void significa vacio o la nada





Las funciones se declaran con un nombre único y deben apegarse a la convención de nombres lowerCamelCase

Implementación de funciones en java

Las funciones deben vivir al nivel de la clase dentro del scope de la class

```java
import java.util.Scanner;

public class Funciones {

    public static void main(String[] args) {

        double y = 3;

        //Area de un circulo
        //pi * r2
        double area = circleArea(y);
        System.out.println(area);

        //Area de una esfera
        //4*PI*r2
        System.out.println(4 * Math.PI * Math.pow(y,2));

        //Volumen de una esfera
        //(4/3)*pi * r3
        System.out.println((4/3) * Math.PI * Math.pow(y,3));
        ////////////////////////////////////////////////////////////////////////////////////////
```


Ssail

```java
        ////////////////////////////////////////////////////////////////////////////////////////
        Scanner leer = new Scanner(System.in);
        String currencySc;
        double quantitySc;
        System.out.println("Ingresa la moneda a convertir en dolar");
        System.out.println("MXN (peso mexicano) o COP (peso colombiano)");
        currencySc = leer.nextLine();
        System.out.println("Ingresa la cantidad a convertir");
        quantitySc = leer.nextDouble();
        System.out.println(convertToDolar(quantitySc, currencySc) + " dolares");
    }

    public static double circleArea (double radio){
        return Math.PI * Math.pow(radio,2);
    }

    public static double sphereArea(double radio){
        return 4 * Math.PI * Math.pow(radio, 3);
    }

    public static double sphereVolumen(double radio){
        return (4/3) * Math.PI * Math.pow(radio,3);
    }

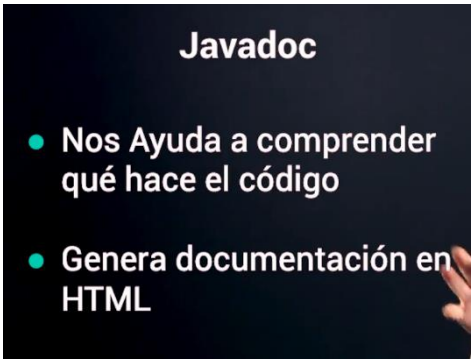
    public static double convertToDolar(double quantity, String currency){
        //MXN COP
        switch(currency){
            case "MXN":
                quantity = quantity * 0.052;
                break;
            case "COP":
                quantity = quantity * 0.00031;
                break;
        }
        return quantity;
    }
```

```
Run:  Funciones
"C:\Program Files\Eclipse Foundation\jdk-11.0.12.7-hotspot\bin\java.exe" "-javaage
28.274333882308138
113.09733552923255
84.82300164692441
Ingresa la moneda a convertir en dolar
MXN (peso mexicano) o COP (peso colombiano)
MXN
Ingresa la cantidad a convertir
200
10.4 dolares

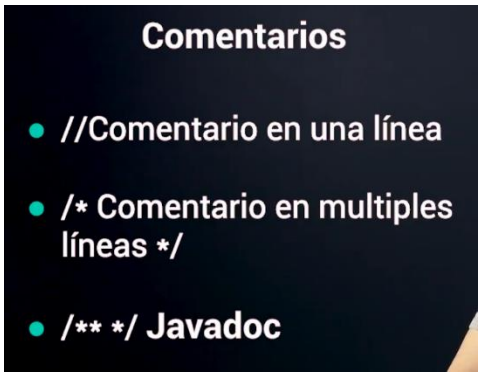
Process finished with exit code 0
```

JAVA DOCS

Esta documentación sirve para cuando se esta trabajando en equipo o cuando se tiene que entregar la documentación a la compañía que nos contrato



Utilizando comentarios es como se puede declarar el JAVA DOC
Los primeros dos el compilador los ignora



Sin embargo los comentarios JAVA DOCS el compilador los lee y genera los java docs.

Para crearla se denomina la descripción de la función, después se pueden utilizar diferentes tags, para este caso utilizamos @param la cual especifica los parámetros que se van a utilizar, también @return que especifica que dato es el que nos va a devolver.

```java
        System.out.println("MXN (peso mexicano) o COP (peso colombiano)");
        currencySc = leer.nextLine();
        System.out.println("Ingresa la cantidad a convertir");
        quantitySc = leer.nextDouble();
        System.out.println(convertToDolar(quantitySc, currencySc) + " dolares");
    }

    public static double circleArea (double radio){
        return Math.PI * Math.pow(radio,2);
    }

    public static double sphereArea(double radio){
        return 4 * Math.PI * Math.pow(radio, 3);
    }

    public static double sphereVolumen(double radio){
        return (4/3) * Math.PI * Math.pow(radio,3);
    }

    /**
     * Descripción: Funcion que especificando su moneda convierte una cantidad de
     * dinero a dolares
     *
     * @param quantity Cantidad de dinero
     * @param currency Tipo de moneda: solo acepta MXM o COP
     * @return quantity Devuelve la cantidad actualizada en dolares
     * */
@   public static double convertToDolar(double quantity, String currency){
        //MXN COP
        switch(currency){
            case "MXN":
                quantity = quantity * 0.052;
                break;
            case "COP":
                quantity = quantity * 0.00031;
                break;
        }
        return quantity;
    }
}
```


Ssail

Activando JAVA DOCS

Para el SO Windows

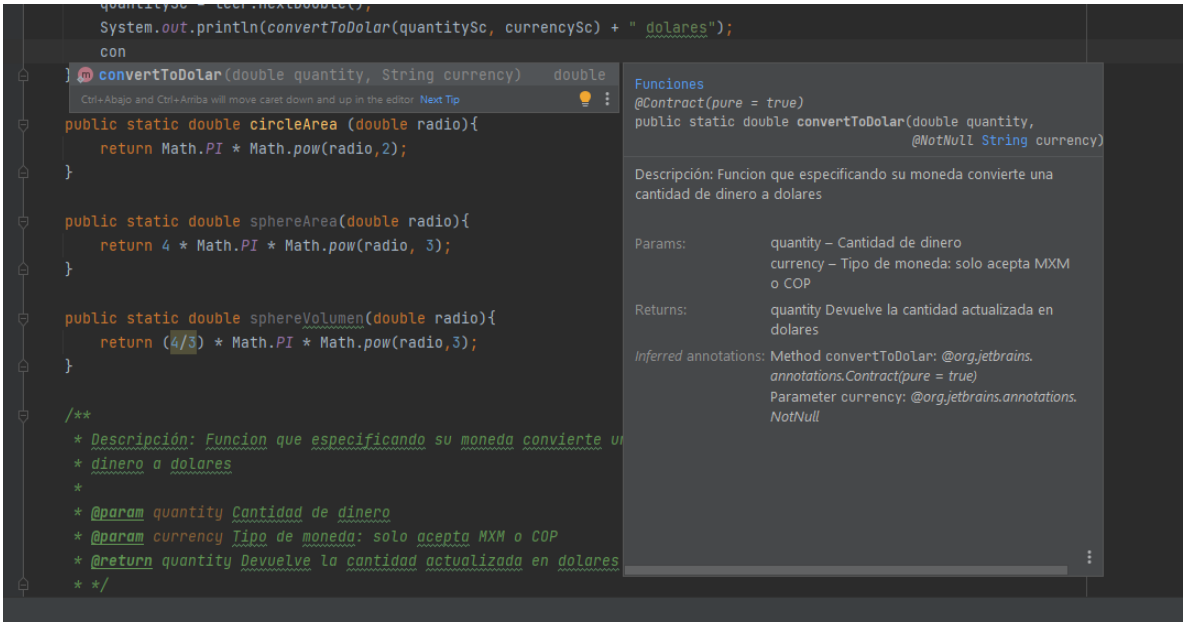
```
File > Settings > Editor > General > Code Completion
```

y pasas a activar la opción

```
"Show the documentation popup in 1000 ms"
```

y por ultimo le das a OK

Y este es el resultado



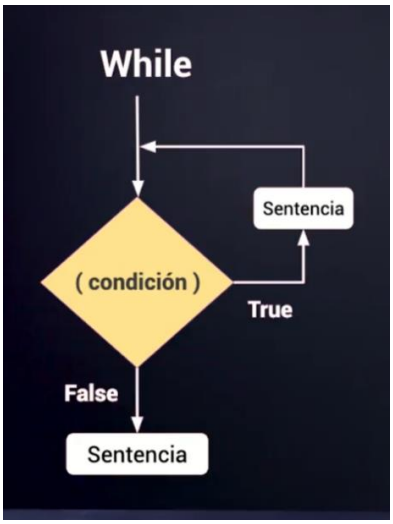
Para encontrar los java docs de otros solo te tienes que posicionar sobre la función y puedes ver sus docs

TAGS DE JAVA DOCS

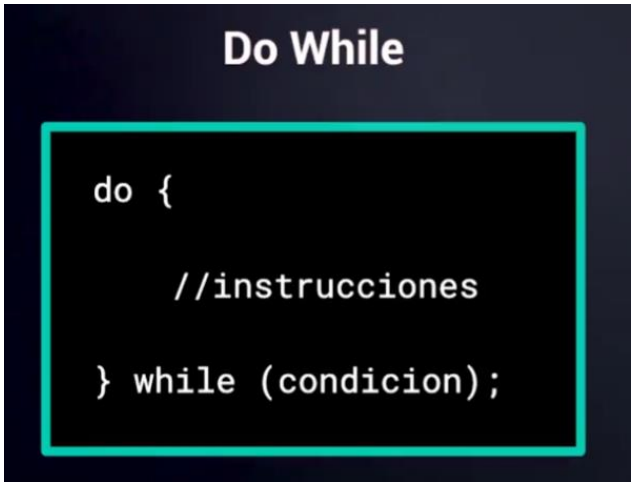
Te dejo algunos tags que puedes utilizar para tus descripciones

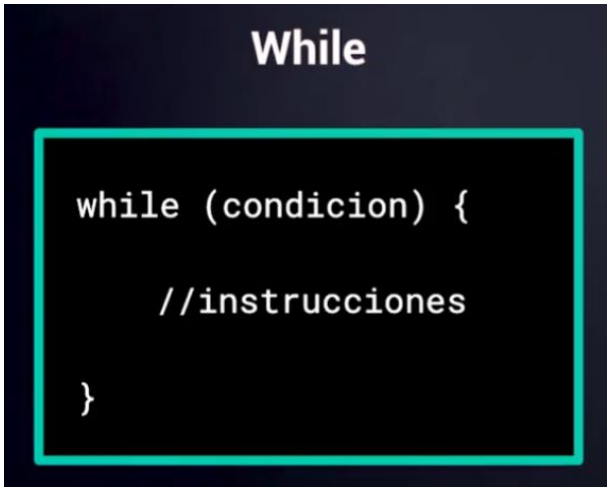
Syntax	Description	Description
@author	Es el autor de la clase	@author name-text
{@code}	Muestra texto en formato de código sin que sea interpretado como código HTML	{@code text}
{@docRoot}	Indica la ruta relativa al directorio raíz del documento generado desde cualquier página generada.	{@docRoot}
@deprecated	Se pone indicando que esta API no debe usarse más	@deprecated deprecatedtext
@exception	Se indica cuando es vulnerable a lanzar una excepción, en seguida se ponen las clases de las excepciones posibles	@exception class-name description
{@inheritDoc}	Indica la herencia o implementación procedentora	
{@link}	Hace un enlace a la miembro indicado	{@link package.class#member label}
{@linkplain}	Lo mismo que en anterior excepto que la etiqueta del enlace se muestra en texto plano	{@linkplain package.class#member label}
@param	Añade parámetros con nombres específicos, seguido de su descripción.	@param parameter-name description
@return	Asigna un parámetro de retorno, seguido de su descripción	@return description
@see	Se añade para manejar referencias, o información relacionada	@see reference
@serial	Se utiliza para indicar campos o clases serializables.	@serial field-description include exclude
@serialData	Se utiliza para documentar métodos que generan una serialización	@serialData data-description
@serialField	Se utiliza para documentar objetos serializados	@serialField field-name field-type field-description
@since	Se añade en el encabezado para especificar desde cuándo fue creado	@since release
@throws	Es sinónimo con la etiqueta @exception	@throws class-name description
{@value}	Cuando es usado sin argumento se usa para especificar un campo estático en otro caso se usa para mostrar el valor constante.	{@value package.class#field}
@version	Se añade en el subtítulo con la versión especificada.	@version version-text

Bucle Do While

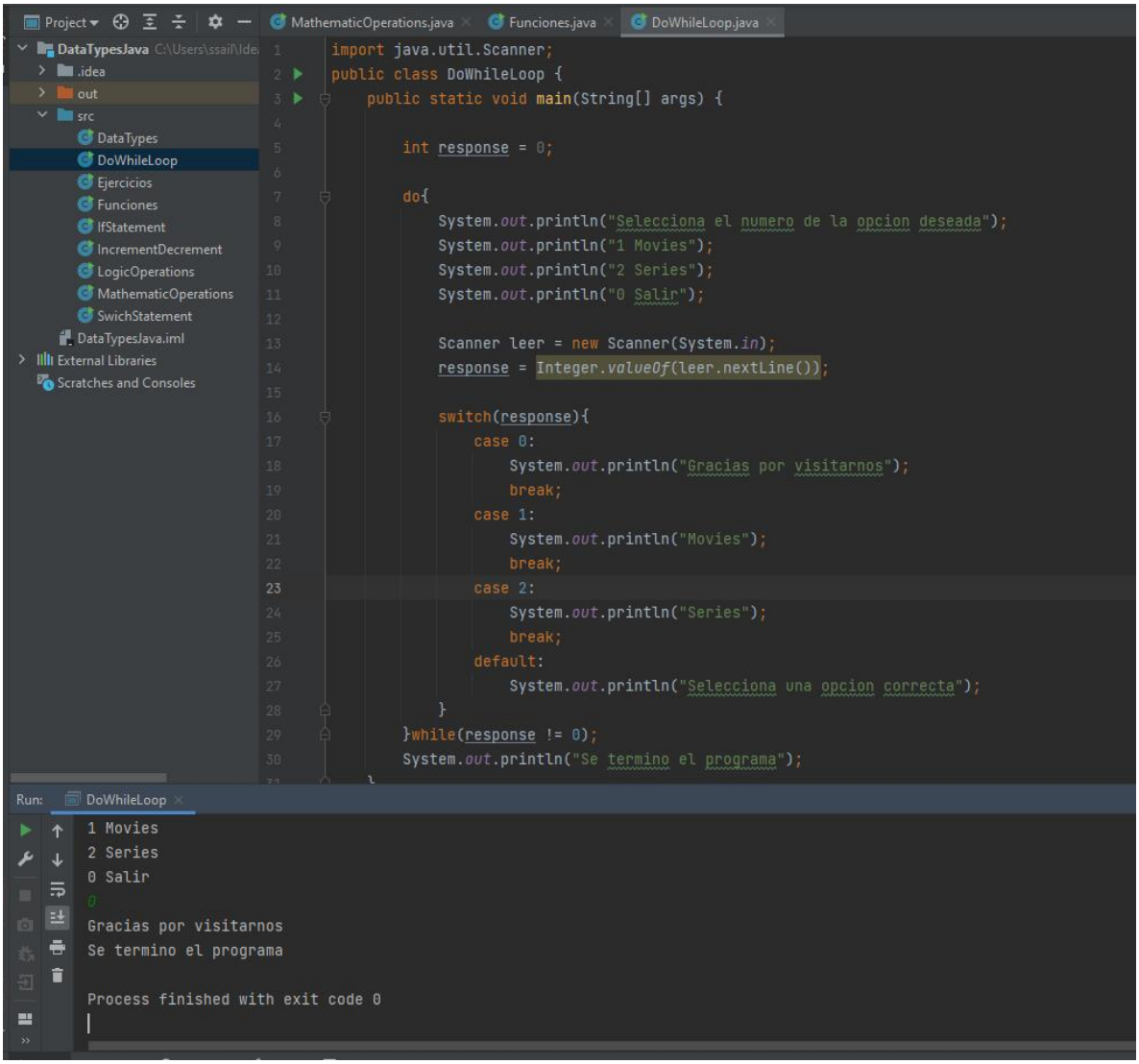


Con do while va a entrar al ciclo y después va a comparar la operación y si la condición se cumple entrar por segunda vez y si no se cumple entro al menos una vez

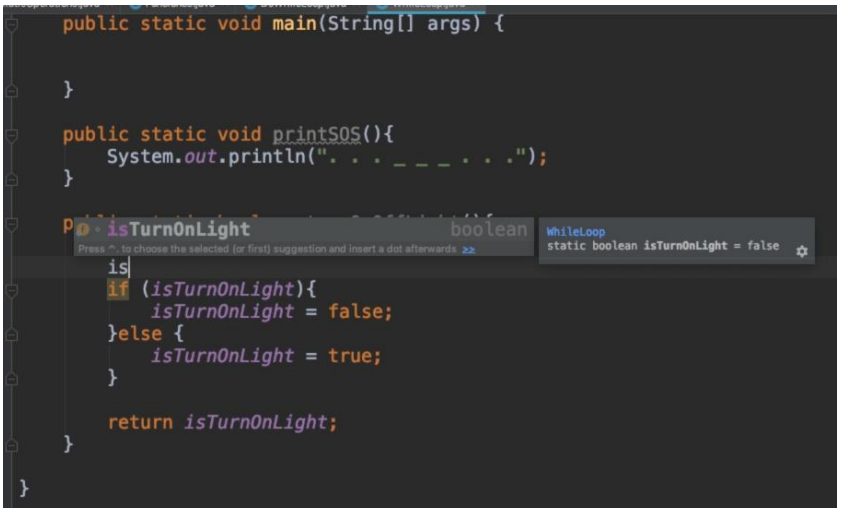




Con while nunca va a entrar al ciclo hasta que la condición se cumpla


Project
DataTypesJava C:\Users\ssail\Ide
.idea
out
src
DataTypes
DoWhileLoop
Ejercicios
Funciones
IfStatement
IncrementDecrement
LogicOperations
MathematicOperations
SwichStatement
DataTypesJava.iml
External Libraries
Scratches and Consoles
MathematicOperations.java
Funciones.java
DoWhileLoop.java
import java.util.Scanner;
public class DoWhileLoop {
public static void main(String[] args) {
int response = 0;
do{
System.out.println("Selecciona el numero de la opcion deseada");
System.out.println("1 Movies");
System.out.println("2 Series");
System.out.println("0 Salir");
Scanner leer = new Scanner(System.in);
response = Integer.valueOf(leer.nextLine());
switch(response){
case 0:
System.out.println("Gracias por visitarnos");
break;
case 1:
System.out.println("Movies");
break;
case 2:
System.out.println("Series");
break;
default:
System.out.println("Selecciona una opcion correcta");
}
}while(response != 0);
System.out.println("Se termino el programa");
Run:
DoWhileLoop
1 Movies
2 Series
0 Salir
0
Gracias por visitarnos
Se termino el programa
Process finished with exit code 0

OPERADOR TERNEARIO



Una manera más óptima al emplear un if, seria con el operador ternario

```
isTurnOnLight = ()?valor:valor;
```

Como primera instancia se escribe la variable, luego entre paréntesis la operación que se va a ejecutar, seguido de un signo de interrogación para indicar que es una operación, luego el primer dato será el valor de regreso seguido de dos puntos para indicar el segundo valor de regreso.

Si la condición se cumple se devuelve el primer dato, si no se cumple se devuelve el segundo dato.

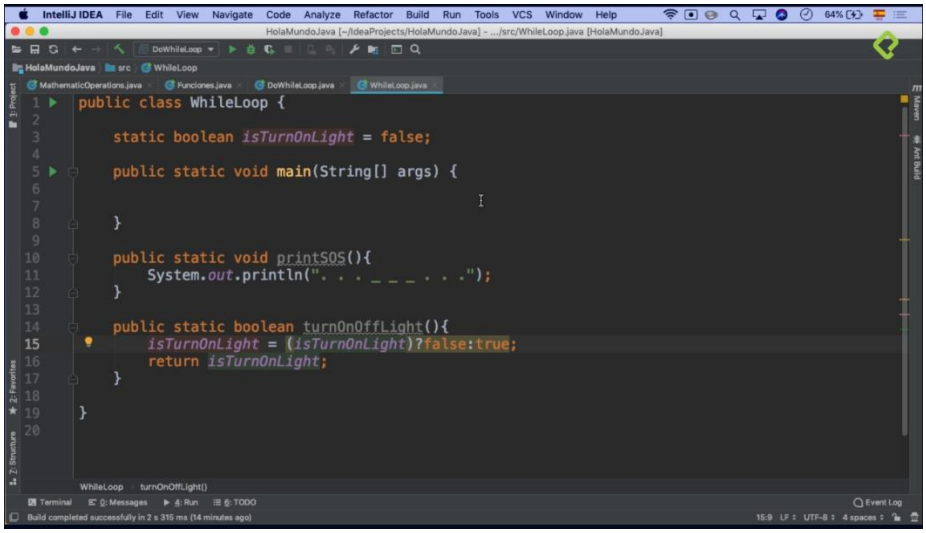
Para este caso, si la variable está encendida (True), le asigna un valor falso a la variable, o si la variable está apagada le asigna un valor verdadero a la variable.

```
isTurnOnLight = (isTurnOnLight)?false:true;
```

De esta manera no ahorramos líneas de código y optimizamos el programa

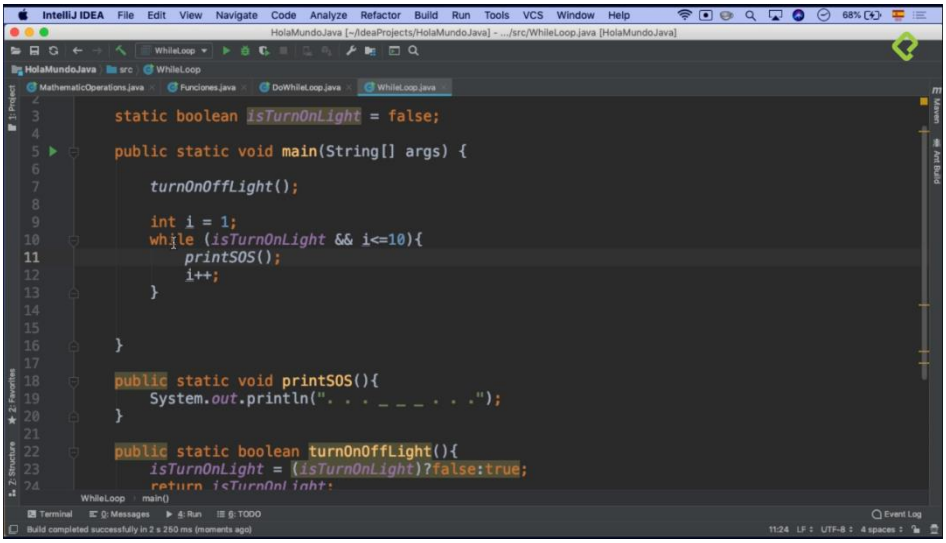
return (isTurnOnLight) ? false : true;
para retornar en la función?

También pudimos escribir de esta manera para retornando directamente la operación



Un ejemplo de Operador Ternario
String TipoVehiculo = (NumRuedas>2)? "Auto" : "Moto";

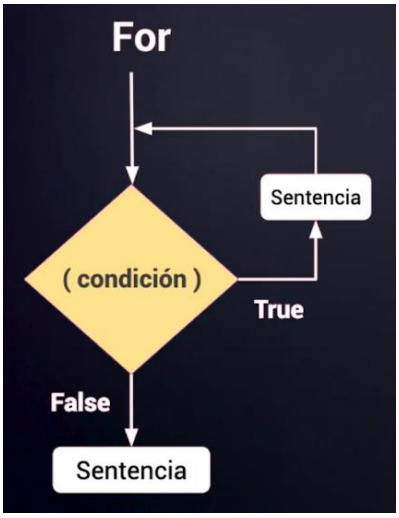
Este es otro ejemplo de cómo podemos utilizar el operador ternario

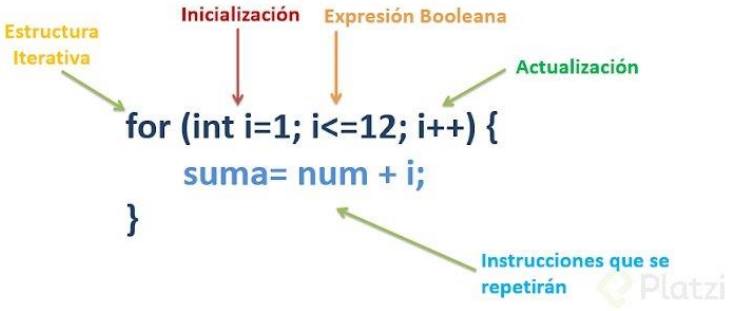


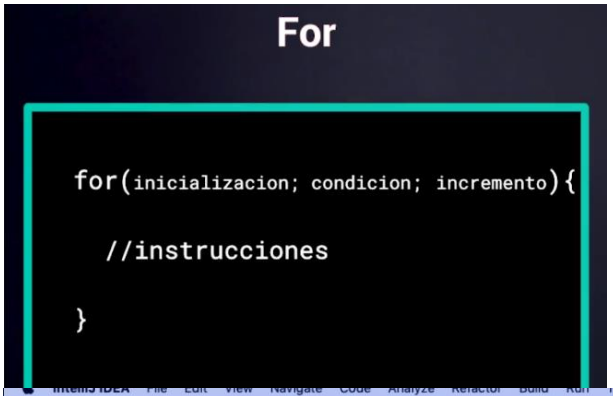
Queda de esta manera

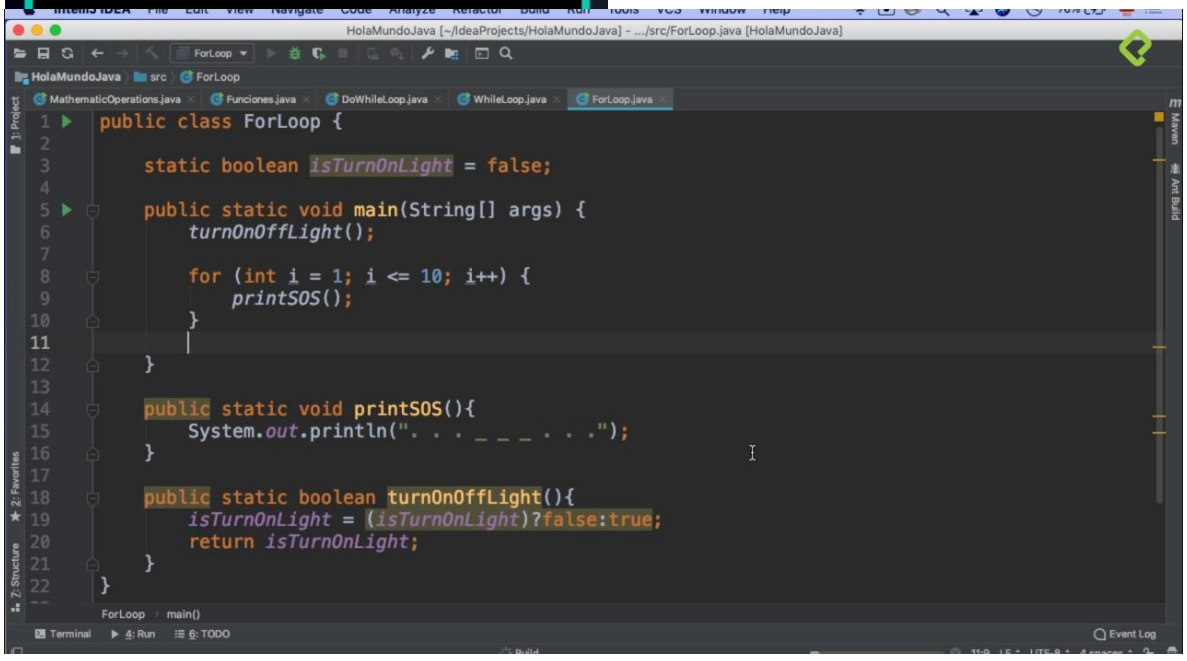
Ssail

Ciclo FOR









Ssail

BREAK, CONTINUE and RETURN

Antes de pasar a uno de nuestros temas más importantes del curso es importante que sepas todas las opciones que tienes para detener ciclos y así seguir controlando el flujo de tus programas.

Break

En Java esta sentencia la verás en dos situaciones especificamente:

1.En un Switch: en esta situación break hace que el flujo del switch no continúe ejecutándose a la siguiente comparación, esto con el objetivo de que solo se cumpla una sola condición:

```
switch (colorModeSelected){
        case "Light":
                System.out.println("Seleccionaste Light Mode");
                break;
         case "Night": //Ambar
                System.out.println("Seleccionaste Night Mode");
                break;
         case "Blue Dark":
                System.out.println("Seleccionaste Blue Dark Mode");
                break;
}
```

2. Para salir de un bucle: Como acabamos de ver un break es capaz de detener el flujo en el código, en este caso detendremos el ciclo como tal terminándolo y haciendo que saltemos a la siguiente instrucción después del ciclo.

Continue

Continue en cierto modo también nos va a servir para detener un ciclo, pero en lugar de terminarlo como en el caso de break, este volverá directo a la condición.

Return

Aunque en algunos lenguajes esta sentencia sirve como un tipo goto, dónde se rompe el flujo del programa la mejor forma de usarlo en Java es en Funciones, cuando lo usamos aquí siempre viene acompañado de un valor, el cual indica el dato que se estará devolviendo en la función.

Notes

`break;` para el loop
`continue;` para la iteración
`return val;` regresa un dato (generalmente de una función)

Ejemplos

Break

```
for (byte i = 0; i<=10; i++) {
        if (i == 4) {
                break;
        }
        System.out.println("i: " + i);
}
```

Este for-i loop va a hacer que cuando `i` sea igual a 4, el loop se va romper.

output:

```
0
1
2
3
```

Continue

```
for (byte i = 0; i<=10; i++) {
        if (i == 4) {
                continue;
        }
  System.out.println("i = " + i);
}
```

Este for-i loop va a imprimir los numeros del 0-9, pero cuando llegue al 4 (i = 4), va a partir la iteración, por ende todo lo que esté después del `continue;` y dentro del scope del for-i loop, no va a ser ejecutado.

output

```
0
1
2
3
5
6
7
8
9
```

Como se pueden dar cuenta, falta el 4, y esto es gracias al `continue` :^)

```
for (int i = 0; i < 10 ; i++){ if(i = 5){ break; // continue; }
System.out.print(i); }
```

si utilizamos break, el programa imprimirá los numero de el 0 al 4
si utilizamos continue, el programa imprimirla los numero del 0 al 9, excepto el 5.

Se me ocurre imprimir los números pares hasta un rango haciendo uso de un **for** e implantando la sentencia **continue**.

```
for (int i = 1; i <=10; i++) {

            if (i%2 !=0) {

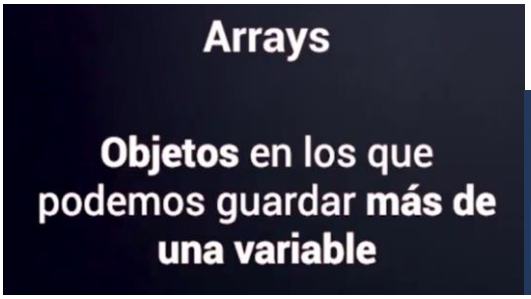
                continue;

            }
            System.out.println(i+", es par");

        }
2, es par
4, es par
6, es par
8, es par
10, es par
```


Ssail

Arrays (Arreglos)



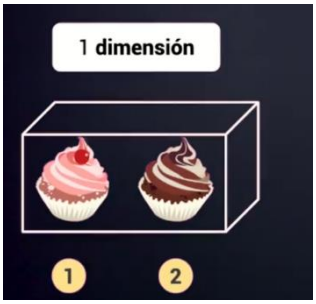


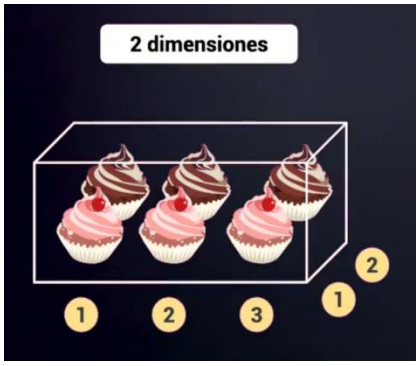
Nos permiten almacenar mas de un elemento

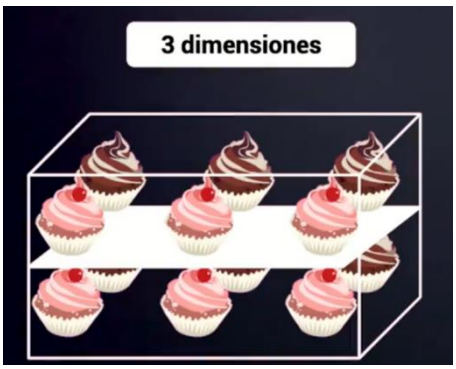
Podemos tener arreglos de dimensiones

1 Dimencion

Elementos uno tras otro en una misma ilera

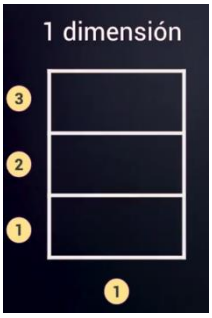






Los que son de 3 dimensiones son como los eje x,y,z, sin embargo existen arreglos con muchas mas dimensiones

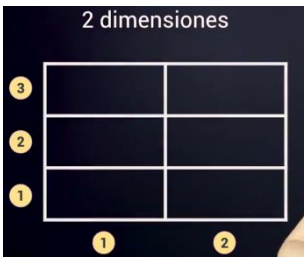
Otra forma de visualizar los arreglos es



Un espacio y tres

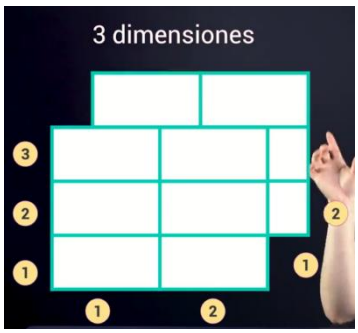
localidades o 3

espacios en esa columna

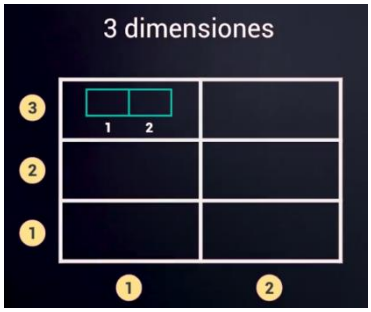


Esta es con dos compartimiento o dos dimenciones

Ssail

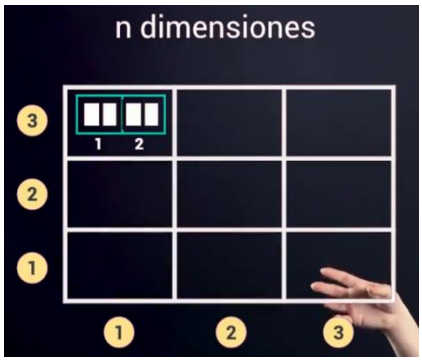


Esto es como paginas de un libro

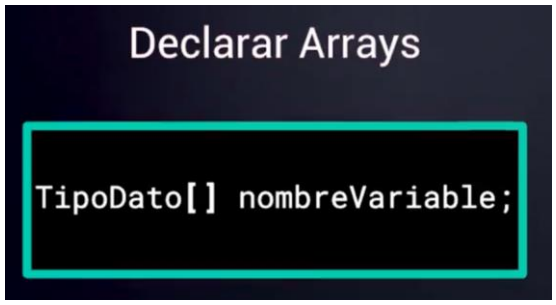


O esta que es como si tuviéramos una caja dentro de otra caja dentro de otra caja (Cajas anidadas)

La primera dimencion va a ser una columna, la segunda dimencion sera una tabla y la tercera dimencion serian cajas anidadas

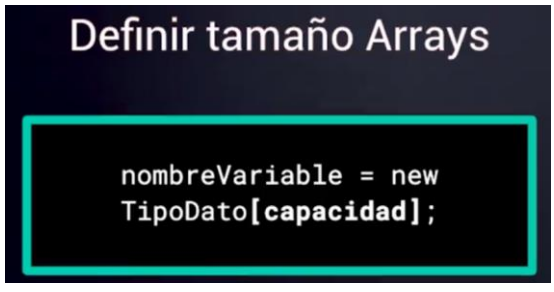


Donde la caja azul también contiene cajas anidadas



Los corchetes nos indican que esa variable que esta ahí es un arreglo y también se puede encontrar asi.

```
TipoDato nombreVariable[];
```



Es importante definir el tamaño de los arreglos, la capacidad se define por los espacios que vamos a utilizar

New

Una vez definida se comienza con la palabra clave new, y esta palabra clave **siempre se va a utilizar cuando estemos utilizando OBJETOS** en java por tanto **los Arrays son objetos**.

Aplicando arreglos

Al declarar la convención de los nombres de los arreglos debe ser en plural

Declarando arrays de una dimensión

Existen 17 versiones de Android en ese momento así que es viable realizar un arreglo sin embargo no es tan eficiente ya que seguirán incrementando las versiones de Android.

En el de los días si es eficiente ya que sabemos que solo existen 7 días y es algo que no cambiara

```
public class Arrays {
    public static void main(String[] args) {
        String[] androidVersions = new String[17];
        String days[] = new String[7];
```

Declarando arrays de dos dimensiones

Lo mas fácil de entenderlos es a manera de una tabla **Matriz** donde el primer corchete representa la primera dimensión pero no es que represente la primera columna o que el segundo corchete represente la segunda columna.

Entonces aclarado el punto, el primer corchete representa los renglones como (México, Mexicp, Colombia, Colombia) y el segundo elemento son las columnas, donde la primera columna esta representada por Country y la segunda por City, aunque Country y City no existen ya que este renglón solo es completamente illustrativo.

```
public class Arrays {
    public static void main(String[] args) {
        String[] androidVersions = new String[17];
        String days[] = new String[7];

        String[][] cities = new String[4][2]; //4 * 2 = 8
        /*
         * +-----------------------------+
         * | Country   |  City           |
         * ------------------------------
         * | México    | CDMX            |
         * | Méxicp    | Guadalajara     |
         * | Colombia  | Bogotá          |
         * | Colombia  | Medellín        |
         * +-----------------------------+
         * */

    }
}
```

Este arreglo tiene una dimensión de 8 elementos.

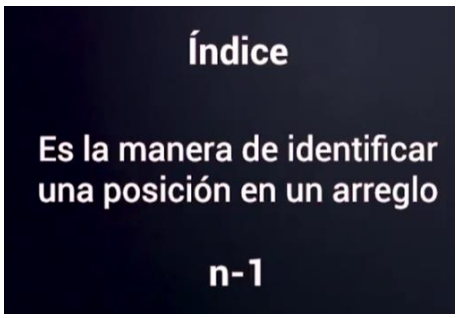
Declarando arrays de 3 en adelante dimensiones

Toma en cuenta que en cuanto empezamos a definir arreglos de 3 dimensiones en la parte practica de manejo de datos es mejor llevarlo hacia una base de datos, sin embargo si hay momentos en los que si se necesita este tipo de arreglos, como cuando usamos cálculos numéricos donde se usen matrices o gráficos en 3 dimensiones

Dos renglones, dos columnas y dos localidades (paginas o cajas)

```
int [][][] numbers = new int[2][2][2];
int [][][][] numbers4 = new int[2][2][2][2];
```

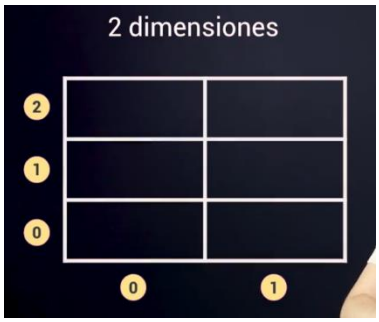
Insertando datos en los arreglos; INDICES Y BUSQUEDA DE ELEMENTOS EN ARRAYS



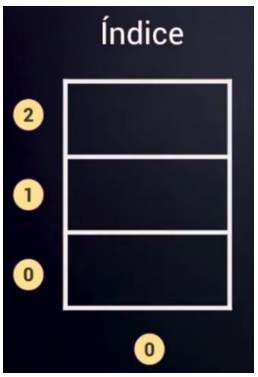
Son variables simples y siempre van a ser enteras

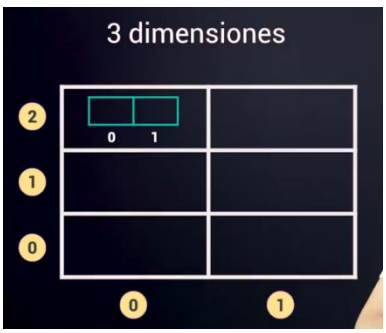
n-1 es la unidad de medida de los índices, ejemplo, si contiene 5 elementos su posición ser 5 - 1 = 4, el índice va a llegar hasta 4

Siempre inician en 0 hasta n-1

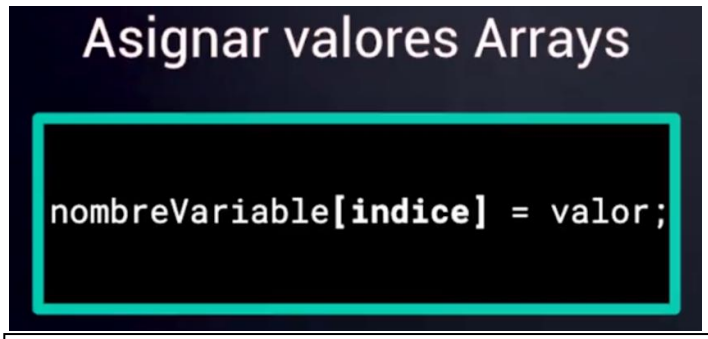


Este arreglo de 1 dimensión, cuya posición de esta dimensión es 0 y si yo quiero ver la posición de los renglones, siempre ira de abajo hacia arriba porque esto es una estructura de datos. Esto simboliza una pila ya que si quieres colocar un nuevo elemento, ese elemento siempre se añadirá en la parte superior, por tanto los índices siempre irán de abajo hacia arriba o y cuando se expanden lo hacen de izquierda a derecha.





Esto es para los arrays de una dimension



Primero lo agregaremos a

```
String[] androidVersions = new String[17];
```

De esta manera se le asignan datos al arreglo

```
androidVersions[0] = "Apple Pie";
androidVersions[1] = "Bannana Bread";
androidVersions[2] = "Cupcake";
androidVersions[3] = "Donut";
```

Para acceder a los datos se realiza de esta manera

```
System.out.println(androidVersions[0]);
System.out.println(androidVersions[1]);
System.out.println(androidVersions[2]);
System.out.println(androidVersions[3]);
```

```
Run:  Arrays
Apple Pie
Bannana Bread
Cupcake
Donut

Process finished with exit code 0
```

Para los arrays de dos dimensiones

```
String[][] cities = new String[4][2]; //4 * 2 = 8
/*
 * +-------------------------+
 * | Country   |  City       |
 * +-------------------------+
 * | México    | CDMX        |
 * | México    | Guadalajara |
 * | Colombia  | Bogotá      |
 * | Colombia  | Medellín    |
 * +-------------------------+
 * */
```

Para esto en el primer corchete se coloca el renglón y en el segundo la columna

```
cities[0][0] = "Colombia";
cities[0][1] = "Medellín";
cities[1][0] = "Colombia";
cities[1][1] = "Bogotá";
cities[2][0] = "México";
cities[2][1] = "Guadalajara";
cities[3][0] = "México";
cities[3][1] = "CDMX";
```

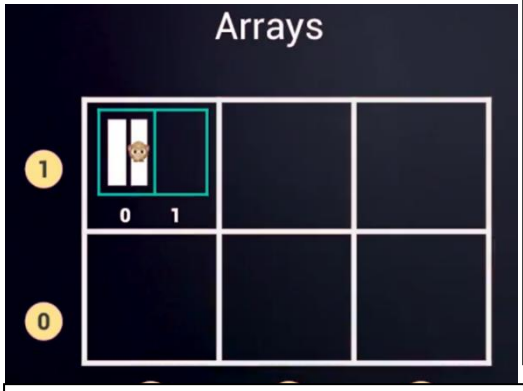
```
Arrays
Colombia
Bogotá
México
Guadalajara
México
CDMX
```

```
System.out.println(cities[0][0]);
System.out.println(cities[0][1]);
System.out.println(cities[1][0]);
System.out.println(cities[1][1]);
System.out.println(cities[2][0]);
System.out.println(cities[2][1]);
System.out.println(cities[3][0]);
System.out.println(cities[3][1]);
```


Ssail

Ejercicio

Trata de localizar la posición donde se encuentra el changuito xd



Nos podemos dar cuanta que son 4 dimenciones
Evaluamos desde la tabla 4 o dimencion 4
Sen encuentra en el renglón 1
Columna 0
Adentro esta en el renglón 0
Y mas adentro esta en la columna 1

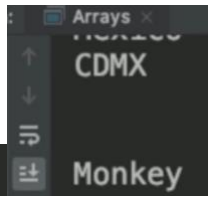
Quedaria asi [1][0][0][1]

En código se vería asi

En primer instancia la tabla 4 tiene 2 renglones y se coloca en el primer corchete, después observamos que tiene 3 columnas y se coloca en el segundo [], dentro tiene dos elementos y se coloca en el tercer [] y dentro de este también existen dos elementos que se colocan en el cuarto [].

```
String[][][][] animals = new String[2][3][2][2];
animals[1][0][0][1] = "Monkey";
```

```
System.out.println(animals[1][0][0][1]);
```



Ssail

Ciclos FOR anidados

Podemos realizar la búsqueda de los anfroid versión con un ciclo for

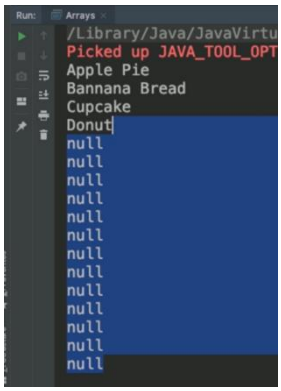
```
String[] androidVersions = new String[17];
androidVersions[0] = "Apple Pie";
androidVersions[1] = "Bannana Bread";
androidVersions[2] = "Cupcake";
androidVersions[3] = "Donut";

System.out.println(androidVersions[0]);
System.out.println(androidVersions[1]);
System.out.println(androidVersions[2]);
System.out.println(androidVersions[3]);
```

Para realizar la impresión de los datos de andriodVersion podemos reducir esas líneas de código con un ciclo for de esta manera

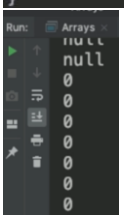
```
                                    //17
for (int i = 0; i < androidVersions.length; i++) {
    System.out.println(androidVersions[i]);
}
```

El .lenght indica la cantidad de elementos que contiene, ya que si lo dejamos así recordemos que le asignamos un vector de 17 elementos, sin embargo, como el índice de for comienza en 0 realizara un incremento de mas en el bucle, también podemos realizar modificaciones para poder trabajar directamente con el arreglo.



Como podemos observar al correr el programa podemos visualizar que recorre cada espacio del arreglo, pero como no tenemos asignados todos los espacios nos muestra los mensajes de null indicándonos que se encuentran vacíos estos lugares

```
int days[] = new int[7];
for (int i = 0; i < days.length; i++) {
    System.out.println(days[i]);
}
```



La inicialización va a depender del tipo de dato que estemos corriendo, en el caso anterior los datos eran string, pero si imrimimos un dato int la inicialización será en 0.

Ahora vamos con los arrays de dos dimensiones.

```
cities[0][0] = "Colombia";
cities[0][1] = "Medellín";
cities[1][0] = "Colombia";
cities[1][1] = "Bogotá";
cities[2][0] = "México";
cities[2][1] = "Guadalajara";
cities[3][0] = "México";
cities[3][1] = "CDMX";
```

```
for (int i = 0; i < cities.length; i++) {
    for (int j = 0; j < cities[i].length; j++) {
        System.out.println(cities[i][j]);
    }
}
```

Donde i es para los renglones y j para las columnas

```
Colombia
Medellín
Colombia
Bogotá
México
Guadalajara
México
CDMX
```

Arrays con más dimensiones

```
String[][][][] animals = new String[2][3][2][2];
animals[1][0][0][1] = "Monkey";

System.out.println();
System.out.println();
System.out.println(animals[1][0][0][1]);

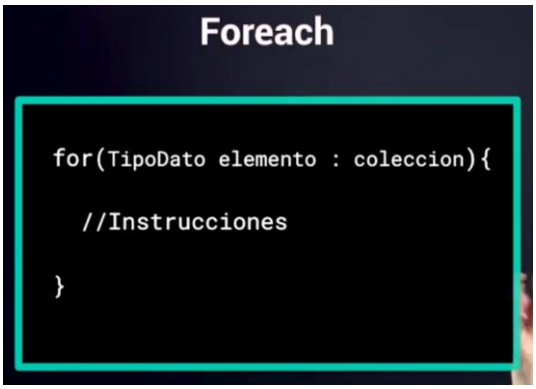
for (int i = 0; i <= 1; i++) {
    for (int j = 0; j <=0 ; j++) {
        for (int k = 0; k <= 0 ; k++) {
            for (int l = 0; l <= 1 ; l++) {
                System.out.println(animals[i][j][k][l]);
            }
        }
    }
}
```

main()

```
Arrays
Monkey
null
null
null
Monkey
```


Ssail

FOREACH



1 dimencion

```
int [][][][] numbers4 = new int[2][2][2][2];

androidVersions[0] = "Apple Pie";
androidVersions[1] = "Bannana Bread";
androidVersions[2] = "Cupcake";
androidVersions[3] = "Donut";

                              //17
/*for (int i = 0; i < androidVersions.length; i++) {
    System.out.println(androidVersions[i]);
}*/

for (String androidVersion : androidVersions) {
    System.out.println(androidVersion);
}
```

Es colo el fori, sin embargo este itera a la varible que creamos despues de declarar el tipo de dato (String) y despues de los dos puntos se coloca el nombre del array o colección de donde sacaremos los datos que se iran guardando en la variable que creamos (androidVersion)

Foreach anidados

Esto es para

```
        for (String[] pair : cities) {
            for ( String name :pair) {
                System.out.println(name);
            }
        }
Arrays > main()
Arrays × Arrays ×
null


Colombia
Medellín
Colombia
Bogotá
México
Guadalajara
México
CDMX
```


Ssail

Consejos For Anidados

Así podemos evitar que se impriman esos “null” en los arreglos:

```
for (int i = 0; i < androidVersions.length; i++) {
            if (androidVersions[i] == null){
                continue;
            }
            System.out.println(androidVersions[i]);
        }

        for (String androidVersion: androidVersions) {
            if (androidVersion == null){
                continue;
            }
            System.out.println(androidVersion);
        }
```

Como pueden ver, si le indicamos que, si la posición del arreglo está vacía, que se la salte para así pasar a la siguiente

Talvez…

Creo que mejor seria un break; en vez de un continue;

```
        System.out.println("Printing the array's values using a nested for");
        for (int i = 0; i < animals.length ; i++) {
            for (int j = 0; j < animals.length ; j++) {
                for (int k = 0; k < animals.length ; k++) {
                    for (int l = 0; l < animals.length ; l++) {
                        if (animals [i][j][k][l] == null){
                            continue;
                        } else {
                            System.out.println(animals[i][j][k][l]);
                        }
                    }
                }
            }

        }
```

```
DoWhileLoop × Arrays ×
Guadalajara
Tampico
Nayarit
Printing the array's values using a nested for
Monkey

Process finished with exit code 0
```